

user-lifecycle

역할: 사용자 identity, 그룹, 포트, 컨테이너와 개인 Kerberos/NFS 준비를 하나의 생명주기로 관리한다.

이 문서는 user-lifecycle 문서의 시작점이다. **설계**는 이 모듈이 왜 중심 역할을 하는지, 디렉터리 구조, 코드 구조, 데이터 모델과 container 생성·Kerberos 흐름을 설명하고, **운영**은 실제 CLI 사용법, 설정과 secret 관리, 테스트와 변경 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	디렉터리 지도, 코드 구조, 데이터 모델, container 생성·Kerberos 활성화 흐름
운영	CLI 사용법, 설정과 secret, post action과 생성물, 테스트, 변경 가이드, 운영 안전 수칙

처음 보는 사람은 **설계 문서**에서 이 모듈이 어떤 자원을 하나의 생명주기로 묶고 왜 공개 CLI를 통과해야 하는지 먼저 확인하고, 실제로 container를 생성·삭제·연장하거나 설정을 바꿔야 할 때 **운영 문서**로 이동한다.

user-lifecycle 설계

개요 · 운영

역할: 사용자 identity, 그룹, 포트, 컨테이너와 개인 Kerberos/NFS 준비를 하나의 생명주기로 관리한다.

1. 이 모듈이 중심인 이유

DECS 사용자 컨테이너는 Docker object 하나가 아니다. 다음 자원이 서로 같은 identity와 상태를 가져야 하는 분산 작업이다.

- 도메인별 UID MySQL의 사용자, 그룹, 예약 ID, container와 port record
- FARM/LAB target host의 Docker container
- NAS 또는 LAB storage의 사용자 home
- 선택적인 AD RFC2307 user/group과 membership
- host의 root-only user keytab과 ccache refresh timer

- 생성·삭제·연장 안내 메일, DB backup과 Excel export

`user-lifecycle` 는 이 자원의 순서와 실패 처리를 소유한다. 다른 모듈이 DB에 직접 쓰거나 container 생성 규칙을 복제하면 UID, 포트와 실제 Docker 상태가 어긋나므로 공개 `uidctl` CLI를 통과하는 것이 원칙이다.

2. 디렉터리 지도

경로	상태	핵심 기능
<code>script/uid_manager/</code>	primary	Python CLI, model, DB, service 구현
<code>script/uid_manager/services/</code>	primary	create/delete/extend/sync/cleanup/group use case
<code>script/uid_manager/kerberos/</code>	primary	AD, keytab, ccache, NFS 검증 명령과 path 생성
<code>script/ansible_playbook/</code>	primary	storage/NAS home과 host Kerberos identity 준비
<code>script/tests/</code>	primary	fake runner/repository 기반 회귀 테스트
<code>config/*.example.*</code>	template	DB, email, maintenance, GPU profile, Google, admin 설정 구조
<code>config/*.local.*</code>	secret/local	실제 credential과 운영 override; Git 제외
<code>server_info/servers.jsonl</code>	shared authority	FARM/LAB 서버 주소와 SSH/NAT port inventory
<code>nfs_mysql/</code>	data layer	MySQL image, schema와 server config
<code>docker-compose.yml</code>	data layer	local UID DB service/volume
<code>maintenance/</code>	utility	사용자 삭제, email CSV 갱신, schema migration 등 관리 작업
<code>ad_backup/</code>	operations	AD backup 생성·검증과 timer 설치
<code>legacy/</code>	compatibility	이전 shell 구현; 신규 기능의 기준이 아님
<code>excel_exports/</code>	generated	사용자 export 결과, source가 아님

`legacy/` 를 즉시 삭제하지 않은 이유는 기존 운영 절차의 비교·복구 근거가 필요하기 때문이다. 그러나 새 비즈니스 규칙은 Python service에 추가하고 테스트 가능하게 유지한다.

3. 코드 구조

CLI와 service 분리

`uid_manager/cli.py` 는 argument parsing, config/repository 생성과 결과 출력만 담는다. 실제 규칙은 service class가 소유한다.

CLI	Service	기능
<code>create-container</code>	<code>ContainerCreateService</code>	identity/port/GPU profile 결정, storage/Kerberos, Docker, DB transaction
<code>delete-container</code>	<code>ContainerDeleteService</code>	대상 단일화, Docker 삭제, DB 비활성화/port 정리
<code>extend-container</code>	<code>ContainerExtendService</code>	만료일 검색·변경과 알림
<code>expired-cleanup</code>	<code>ExpiredCleanupService</code>	만료 대상 계획 또는 실제 정리
<code>sync-containers</code>	<code>ContainerSyncService</code>	DB record와 local Docker 상태 비교·재생성 계획
<code>manage-group</code>	<code>GroupManagementService</code>	AD/DB group과 membership 관리

이 분리는 CLI 외의 timer/test에서도 동일한 규칙을 호출하고, fake repository와 recording runner로 production 접속 없이 계획을 검증하기 위한 것이다.

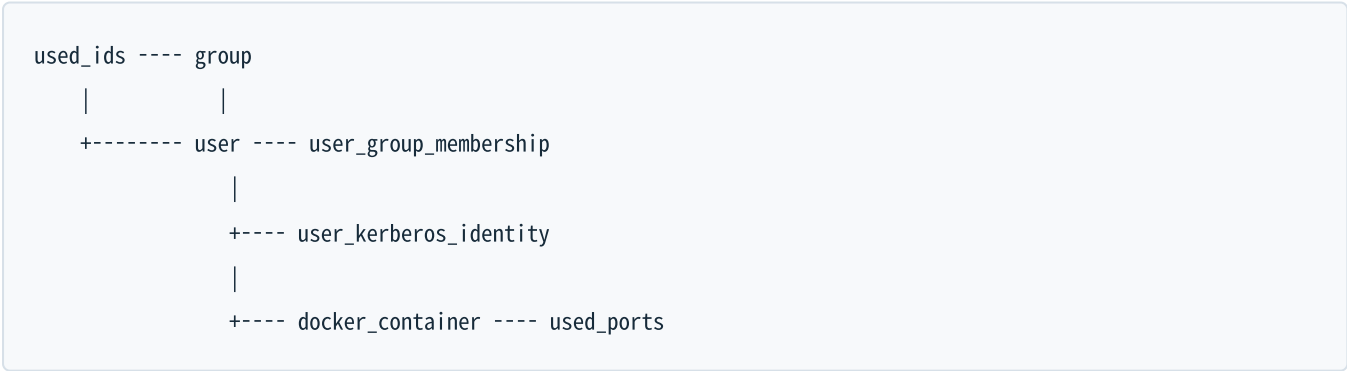
Port와 runner 추상화

`LocalRunner` 와 `AnsibleRunner` 는 subprocess와 원격 shell/playbook 호출을 한 곳에 모은다. `RecordingRunner` 는 실행 대신 command를 기록한다. `PortMapping` 은 host port, container port와 purpose를 함께 보존하여 단순 문자열 port가 SSH/Jupyter/VNC 중 무엇인지 알지 않게 한다.

`OperationPlan` 은 facts, 단계와 명령을 렌더링한다. `--dry-run` 이 실제 실행 경로와 같은 준비 로직을 통과하므로 문서용 예상 명령과 실제 입력 검증이 달라지는 것을 줄인다.

4. 데이터 모델과 권위

MySQL schema의 주요 관계:



데이터	의미
<code>used_ids</code>	UID/GID 숫자의 전역 예약
<code>group</code>	group 이름과 GID
<code>user</code>	실명, username, UID/GID, 연락처
<code>user_group_membership</code>	supplemental/primary group 관계

데이터	의미
<code>user_kerberos_identity</code>	AD realm/SID/RFC2307과 최근 NAS/NFS 관측값
<code>docker_container</code>	image, server, GPU profile, 만료, 생성자와 활성 상태
<code>used_ports</code>	host port, 용도와 container record 연결

foreign key와 unique index를 쓰는 이유는 application 검증만으로 막기 어려운 중복 UID/GID, container와 port 충돌을 DB에서도 거부하기 위해서다. FARM과 LAB은 DB endpoint가 다를 수 있으므로 `AppConfig`가 domain별 host를 선택한다.

5. Container 생성 흐름

5.1 준비와 identity 채택

`ContainerCreateService.prepare()`는 이름, domain/server, 날짜, image와 port를 검증한 뒤 UID/GID source를 결정한다. 신규 사용자의 UID 우선순위는 다음과 같다.

1. DB에 기존 사용자가 있으면 DB UID/GID
2. 운영자가 명시한 `--uid / --gid`
3. Kerberos 모드에서 기존 AD RFC2307 identity
4. 기존 storage home의 숫자 owner
5. 모두 없으면 DB의 다음 사용 가능한 ID

기존 자원을 무조건 새 ID로 덮지 않고 채택하는 이유는 이미 소유권이 있는 NFS home이나 AD object를 고아로 만들지 않기 위해서다. 여러 source가 서로 다른 값을 말하면 자동 보정하지 않고 validation error로 중단한다.

port는 DB 예약과 실제 target host Docker publish port를 함께 확인한다. 기본 SSH/Jupyter, 선택적 VNC와 추가 container port를 배정하거나 `--fixed-port-mappings`로 host:container[:purpose]를 명시할 수 있다.

같은 단계에서 GPU profile도 결정된다. 여러 GPU 모델이 섞여 있는 서버(현재는 `GPU_PROFILE_REQUIRED_SERVERS`에 등록된 LAB5만 해당)는 `--gpu-profile`로 사용할 GPU 묶음을 선택해야 한다. `AppConfig.resolve_gpu_profile()`이 `config/gpu_profiles.local.json` 카탈로그에서 서버별 profile 이름(예: `a6000`, `pro5000`)을 실제 GPU UUID 목록으로 바꾸고, 이 값이 나중에 Docker 실행 시 `--gpus device=<uuid,...>`로 전달된다. 카탈로그에 없는 서버는 GPU 종류가 하나뿐이라고 보고 기본값 `all`(모든 GPU 노출)을 쓴다. 선택된 profile 이름은 `docker_container.gpu_profile`에 저장되어, 이후 `sync-containers`가 DB와 실제 Docker `--gpus` 설정의 drift를 비교하는 기준이 된다.

5.2 실행 순서

입력 검증과 OperationPlan

|

v

target image inspect/pull

|

v

storage home + optional AD/keytab/ccache 준비

|

v

NSS/idmap/실제 NFS write 검증

|

v

docker run + inspect/port 검증

|

v

DB user/group/identity/container/port 단일 transaction

|

v

email + DB backup + export

storage와 Kerberos write 검증을 Docker/DB 확정 전에 두는 이유는 사용자가 로그인했지만 home에 쓸 수 없는 반쪽 container를 만들지 않기 위해서다. Docker 생성 뒤 DB write는 한 transaction으로 묶는다. DB 단계가 실패하면 transaction을 rollback하고 방금 만든 container 정리를 시도하여 실제 상태와 record가 갈라지는 시간을 줄인다.

`--no-db-record`는 특수 검증용으로 Docker를 만들되 DB user/group/container/port write를 생략한다. 일반 운영 생성에는 사용하지 않는다. DB에 없는 container는 GPU user attribution, 만료 정리와 sync에서 정상 관리 대상이 아니기 때문이다.

5.3 domain별 storage

일반 LAB root_squash 경로는 storage server에서 실제 export home을 만들고 선택한 UID/GID로 owner를 설정한 뒤, target host에 mount된 `/home/tako<N>/share/user-share`를 container `/home`에 bind한다.

FARM은 NAS의 user-share 구조를 사용한다. Kerberos가 아닌 모드도 target host root가 NFS root_squash 때문에 직접 owner를 바꾸려 하지 않고 storage/NAS 소유 playbook을 통해 준비한다.

6. Kerberos 활성화 흐름

6.1 공통 보안 모델

```

AD user/private group + RFC2307
  |
  v
root-only keytab on target host
  |
  v
systemd refresh -> /run/user/<uid>/krb5cc
  |
  v
container receives ccache only

```

사용자 password를 DB나 container에 저장하지 않는다. AD에서 keytab을 export한 뒤 target host의 `/etc/decs-krb/keytabs/`에 mode `0400`으로 설치한다. refresh service는 ticket을 renew하거나 만료 여유가 부족하면 keytab으로 재발급하고, container에는 ccache만 전달한다.

keytab rotation은 `--rotate-kerberos-keytab`처럼 명시적으로 요청한다. 정상 container 재생성이나 만료 연장이 credential rotation을 뜻하지 않게 하기 위해서다.

6.2 FARM

FARM 흐름은 AD user/group과 RFC2307을 보장하고 NAS mapping과 home을 준비한 뒤 target host에서 해당 UID와 ccache로 실제 NFS write를 수행한다.

공유 NAS GSS/idmap service restart는 기본적으로 꺼져 있다. 새 identity의 cache 문제가 의심되더라도 NAS service 재시작은 모든 client의 GSS context를 뚫을 수 있으므로 명시적 유지보수 옵션으로만 허용한다.

6.3 LAB

LAB Kerberos PoC/지원 흐름은 LAB Samba AD, storage의 전용 Kerberos export, target host의 NSS/idmap과 NFSv4.1을 함께 확인한다. storage와 client의 NFSv4 idmap domain이 다르면 owner가 `nobody`로 보이거나 `chgrp`가 실패할 수 있다. 따라서 ticket 성공만으로 완료하지 않고 숫자 owner와 실제 write를 검증한다.

6.4 DB에 남기는 Kerberos 정보

DB에는 password/keytab이 아니라 다음 비밀이 아닌 identity metadata만 둔다.

- AD username, realm, NetBIOS domain
- domain/object SID
- AD `uidNumber`, `gidNumber`
- 최근 NAS internal UID/GID
- 최근 NFS에서 본 UID/GID와 검증 시각

SID와 최근 관측값을 보존하면 같은 username이 재생성되었거나 mapping이 달라진 상황을 단순 문자열 일치보다 안전하게 발견할 수 있다.

user-lifecycle 운영

개요 · 설계

1. 주요 CLI 사용법

설치/실행 위치:

```
cd /home/jy/server_manage/user-lifecycle/script
python3 -m pip install -e .
uidctl --help
```

생성 dry-run

```
uidctl create-container \
  --name '홍길동' \
  --username hong \
  --server-id FARM8 \
  --expiration-date 2026-12-31 \
  --image decs \
  --version cuda12.5-tf2.20-ubuntu22.04-260706 \
  --created-by admin \
  --email hong@example.org \
  --phone 000-0000-0000 \
  --dry-run
```

Kerberos/VNC는 필요할 때 명시한다.

```
uidctl create-container ... --enable-kerberos --enable-vnc --dry-run
```

GPU 모델이 여러 종류인 서버(현재는 LAB5)는 `--gpu-profile` 로 사용할 GPU 묶음을 선택한다. 생략하면 서버의 기본 profile이 쓰이고, 그 서버가 `GPU_PROFILE_REQUIRED_SERVERS` 에 등록되어 있는데 카탈로그에 없는 profile을 요청하면 오류로 중단된다.

```
uidctl create-container ... \  
  --server-id LAB5 \  
  --gpu-profile a6000 \  
  --dry-run
```

삭제

DB record 검색 조건으로 대상이 하나인지 먼저 확인하고 dry-run한다.

```
uidctl delete-container \  
  --server-id FARM8 \  
  --container-name hong \  
  --dry-run
```

만료 연장과 정리

연장은 기본적으로 계획만 만들고 실제 변경에 `--apply` 가 필요하다.

```
uidctl extend-container \  
  --username hong \  
  --expiration-date 2027-06-30  
  
uidctl extend-container \  
  --username hong \  
  --expiration-date 2027-06-30 \  
  --apply
```

만료 정리도 대상 날짜와 domain을 제한해 먼저 dry-run한다.

```
uidctl expired-cleanup \  
  --today 2026-07-17 \  
  --domains FARM,LAB \  
  --dry-run
```

sync와 group


```
uidctl sync-containers --domain FARM --dry-run
uidctl manage-group show --domain FARM --group research
uidctl manage-group add-user \
  --domain FARM \
  --group research \
  --user hong \
  --dry-run
```

`--force`, `--auto-delete`, 실제 group delete는 영향 범위를 확인한 뒤 사용한다. `sync-containers` 는 DB의 `gpu_profile` 과 실제 Docker `--gpus` 설정이 어긋나는 경우도 함께 찾아낸다.

2. 설정과 secret

공개 template:

- `config/db_config.example.env`
- `config/email_config.example.env`
- `config/daily_maintenance.example.env`
- `config/google-client.example.json`
- `config/reminder_admins.example.txt`
- `config/gpu_profiles.example.json`
- `ad_backup/config.example.env`

실제 값은 대응하는 `.local.*` 또는 ignored 운영 파일에 둔다. 문서, log, OperationPlan과 exception에 DB password, SMTP credential, Google secret, AD password, keytab 내용이 나타나지 않도록 command redaction을 유지한다.

`config/gpu_profiles.local.json` 은 서버별 GPU profile 이름과 실제 GPU UUID 목록의 카탈로그다.

`config/db_config.local.env` 의 `GPU_PROFILES_FILE` 이 이 파일의 경로를, `GPU_PROFILE_REQUIRED_SERVERS` 가 profile 지정을 강제할 서버 목록을 정한다. GPU UUID 자체는 credential이 아니지만, 실제 서버의 GPU 구성과 어긋나면 엉뚱한 GPU가 배정되므로 하드웨어를 바꿀 때마다 함께 갱신해야 한다.

`config/network_topology.json` 과 `server_info/servers.jsonl` 은 secret 저장소가 아니다. 접근 경로에 필요한 주소와 port만 넣고 password/private key는 외부 SSH/Ansible 설정이 관리한다.

3. Post action과 생성물

성공한 생명주기 작업 뒤 `PostActions` 가 이메일, DB backup과 export 도구를 호출한다. 핵심 transaction과 알림을 분리한 이유는 메일 실패가 identity/DB 일관성을 되돌리는 근거가 되지 않기 때문이다. 반대로 DB transaction이 실패했는데 성공 메일을 보내지 않도록 실행 순서는 transaction 뒤에 둔다.

`excel_exports/`, `AD backups/`, `server_info`의 생성된 raw 정보는 운영 artifact다. source code처럼 수정하거나 매 뉴얼의 권위 있는 입력으로 삼지 않는다. 보존·암호화·삭제 주기는 별도 운영 정책에 따라야 한다.

4. 테스트

```
cd /home/jy/server_manage/user-lifecycle/script
python3 -m unittest discover -s tests -v
```

또는 프로젝트 test runner:

```
python3 tests/run_tests.py
```

중요 회귀 범위:

- 자동/고정 port 할당과 중복 거부
- DB/AD/storage 기존 identity 채택 우선순위
- GPU profile 카탈로그 해석과 필수 서버의 fail-closed 동작
- create 실패 시 DB rollback과 container cleanup
- dry-run에서 원격/DB 변경이 없는지
- FARM/LAB Kerberos command와 path 차이
- group 삭제/primary 변경의 안전 gate
- sync가 DB에 없는/멈춘 container와 GPU profile drift를 구분하는지
- command/log에서 password redaction

shell legacy test는 호환성 확인용이며 Python service test를 대체하지 않는다.

5. 변경 가이드

새 lifecycle command

1. request/result 또는 record를 `models.py`에 정의한다.
2. 비즈니스 순서를 `services/`의 class로 구현한다.
3. DB query는 `Repository` protocol과 `MySQLRepository`에 추가한다.
4. 외부 명령은 runner를 통하고 secret redaction을 적용한다.
5. CLI에는 parsing과 dependency 조립만 추가한다.
6. fake repository/runner를 사용한 dry-run, success와 partial failure test를 작성한다.

schema 변경

- 기존 운영 DB에 idempotent하게 적용되는 migration/ensure path를 준비한다. (예: `maintenance/migrate_add_gpu_profile.sql` 처럼 컬럼 존재 여부를 먼저 확인한다.)
- unique/foreign key와 기존 데이터 backfill 순서를 검토한다.
- FARM/LAB DB를 독립적으로 적용·검증한다.
- exporter가 읽는 column과 query 호환성을 `monitoring` 에서 함께 확인한다.

Kerberos 변경

- password/keytab이 DB, plan과 log에 들어가지 않는지 확인한다.
- DB, AD, NAS/NFS와 container identity 불변 조건을 test한다.
- 공유 NAS service restart를 정상 경로에 추가하지 않는다.
- 운영 policy 변경이면 `kerberos-nfs/docs/farm.md` 또는 `docs/lab.md` 를 갱신한다.

6. 운영 안전 수칙

- 모든 destructive 명령은 domain, server와 대상 container가 하나인지 확인한다.
- 실제 생성 전 `--dry-run` 의 identity source, mount, image, port와 GPU profile을 검토한다.
- 기존 AD 또는 storage identity와 요청 UID/GID 충돌을 강제로 덮지 않는다.
- DB와 실제 Docker 상태가 어긋나면 원인을 확인하고 `sync --auto-delete` 를 바로 사용하지 않는다.
- keytab과 사용자 password를 container나 backup export에 포함하지 않는다.
- GPU profile 카탈로그가 실제 서버의 GPU 구성과 다르면 먼저 카탈로그를 맞추고, DB의 `gpu_profile` 값을 임의로 고쳐 맞추지 않는다.
- `legacy/` 를 수정해 신규 규칙을 우회하지 않는다.
- DB 없는 test container는 이름, 수명과 정리 책임을 명확히 한다.